

Week 4 Structured Notes: 2D Arrays and Memory Mapping

Topic Overview

Week 4 moves from **one-dimensional arrays** to **two-dimensional arrays**.

In Week 1, you learned how to create and use normal arrays.

In Week 2, you learned about stack arrays, heap arrays, pointers, and memory.

In Week 3, you learned how to resize heap arrays and calculate the address of a 1D array element using this formula:

$$\text{Address of } A[i] = L0 + (i * w)$$

Week 4 now extends that idea to **2D arrays**.

This week mainly covers:

1. What a 2D array is
2. Rows and columns
3. Meaning of `A[i][j]`
4. Declaring 2D arrays
5. Initializing 2D arrays
6. Nested loops
7. Printing a matrix
8. Printing one row
9. Printing one column
10. Finding the sum of all elements
11. How 2D arrays are stored in memory
12. Row-major mapping
13. Column-major mapping
14. Row-major address formula
15. Column-major address formula
16. Difference between row-major and column-major storage
17. Three ways to create 2D arrays
18. Common mistakes
19. Practice programs
20. Practice questions

1. What You Should Know Before Week 4

Before learning Week 4, you should already understand these ideas.

From Week 1

You should know that:

- A variable stores one value.
- An array stores many values under one name.
- Array indexing starts from `0` in C and C++.
- A 5-element array has indexes `0` to `4`.
- A loop is used to visit all array elements.
- Array elements are stored side by side in memory.
- Accessing one element is fast.

Example 1D array:

```
int A[5] = {2, 4, 6, 8, 10};
```

Logical view:

Index:	0	1	2	3	4
Value:	2	4	6	8	10

From Week 2

You should know that:

- Stack arrays are fixed-size arrays.
- Heap arrays are created dynamically.
- A pointer stores a memory address.
- `malloc` creates heap memory in C.
- `free` releases heap memory in C.
- `new[]` creates a heap array in C++.
- `delete[]` releases a heap array in C++.

From Week 3

You should know that:

- Arrays need contiguous memory.
- Arrays cannot grow directly.

- Heap arrays can be resized manually by creating a bigger block and copying values.
- The computer calculates an array element address instead of searching for it.
- The 1D address formula is:

Address of $A[i] = L0 + (i * w)$

Where:

$L0$ = base address
 i = index
 w = word size

2. Main Idea of Week 4

Week 4 changes the way we look at arrays.

A 1D array looks like a line:

[10] [20] [30] [40]

A 2D array looks like a table:

1	2	3	4
5	6	7	8
9	10	11	12

So the main upgrade is:

Week 3: $A[i]$
 Week 4: $A[i][j]$

Simple meaning:

$A[i]$ = one index
 $A[i][j]$ = two indexes

A 2D array uses:

```
i = row index  
j = column index
```

3. What Is a 2D Array?

A **2D array** is an array arranged in rows and columns.

It looks like a table or matrix.

Example:

```
A =  
1  2  3  4  
5  6  7  8  
9 10 11 12
```

This array has:

```
3 rows  
4 columns
```

So in C, we write:

```
int A[3][4];
```

Meaning:

```
A has 3 rows and 4 columns.
```

Total number of elements:

```
3 * 4 = 12 elements
```

4. Rows and Columns

A row goes left to right.

A column goes top to bottom.

Example:

	Column 0	Column 1	Column 2	Column 3
Row 0	1	2	3	4
Row 1	5	6	7	8
Row 2	9	10	11	12

Here:

```
Row 0 = 1  2  3  4
Row 1 = 5  6  7  8
Row 2 = 9 10 11 12
```

And:

```
Column 0 = 1 5 9
Column 1 = 2 6 10
Column 2 = 3 7 11
Column 3 = 4 8 12
```

5. Meaning of `A[i][j]`

In a 2D array:

```
A[i][j]
```

means:

```
Go to row i.
Then go to column j.
```

In C and C++, indexing starts from `0`.

So:

```
A[0][0] = row 0, column 0
A[0][1] = row 0, column 1
A[1][0] = row 1, column 0
A[1][2] = row 1, column 2
```

Example array:

```
A =
      Col 0  Col 1  Col 2  Col 3
Row 0    1     2     3     4
Row 1    5     6     7     8
Row 2    9    10    11    12
```

Then:

```
A[0][0] = 1
A[0][1] = 2
A[0][2] = 3
A[0][3] = 4

A[1][0] = 5
A[1][1] = 6
A[1][2] = 7
A[1][3] = 8

A[2][0] = 9
A[2][1] = 10
A[2][2] = 11
A[2][3] = 12
```

Important:

```
A[1][2] does not mean first row, second column.
```

Correct meaning:

```
A[1][2] = row index 1, column index 2
```

Because indexing starts from 0:

```
row index 1    = second row  
column index 2 = third column
```

So:

```
A[1][2] = second row, third column
```

6. Declaring a 2D Array

Declaration means creating the array.

Example:

```
int A[3][4];
```

This means:

```
A is a 2D integer array.  
It has 3 rows.  
It has 4 columns.
```

The first number is the number of rows.

The second number is the number of columns.

```
int A[3][4]  
    ↑  ↑  
    |  |  
  rows columns
```

So:

```
int A[3][4];
```

means:

3 rows and 4 columns

Not:

4 rows and 3 columns

7. Initializing a 2D Array

Initialization means giving starting values to the array.

Example:

```
int A[3][4] = {  
    {1, 2, 3, 4},  
    {5, 6, 7, 8},  
    {9, 10, 11, 12}  
};
```

This creates:

Row 0:	1	2	3	4
Row 1:	5	6	7	8
Row 2:	9	10	11	12

Each inner pair of braces represents one row.

{1, 2, 3, 4}

This is row 0.

{5, 6, 7, 8}

This is row 1.

{9, 10, 11, 12}

This is row 2.

8. Accessing One Element

Accessing means getting one value from the array.

Example:

```
printf("%d", A[1][2]);
```

For this array:

```
A =  
1  2  3  4  
5  6  7  8  
9 10 11 12
```

`A[1][2]` means:

row 1, column 2

Row 1 is:

5 6 7 8

Column 2 in that row is:

7

So:

`A[1][2] = 7`

9. Why 2D Arrays Need Nested Loops

For a 1D array, one loop is enough.

Example:

```
for (int i = 0; i < 5; i++) {  
    printf("%d ", A[i]);  
}
```

A 1D array has only one direction:

index 0, index 1, index 2, index 3, ...

But a 2D array has two directions:

rows
columns

So we normally need two loops.

This is called a **nested loop**.

A nested loop means one loop inside another loop.

Example:

```
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 4; j++) {  
        printf("%d ", A[i][j]);  
    }  
    printf("\n");  
}
```

The outer loop controls the rows.

The inner loop controls the columns.

```
i = row  
j = column
```

10. How the Nested Loop Works

Using this array:

```
A =  
1  2  3  4  
5  6  7  8  
9 10 11 12
```

Code:

```
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 4; j++) {  
        printf("%d ", A[i][j]);  
    }  
    printf("\n");  
}
```

Step-by-step:

When $i = 0$:

```
j = 0 -> A[0][0] = 1  
j = 1 -> A[0][1] = 2  
j = 2 -> A[0][2] = 3  
j = 3 -> A[0][3] = 4
```

When $i = 1$:

```
j = 0 -> A[1][0] = 5  
j = 1 -> A[1][1] = 6  
j = 2 -> A[1][2] = 7  
j = 3 -> A[1][3] = 8
```

When $i = 2$:

```
j = 0 -> A[2][0] = 9  
j = 1 -> A[2][1] = 10
```

```
j = 2 -> A[2][2] = 11
j = 3 -> A[2][3] = 12
```

The output becomes:

```
1 2 3 4
5 6 7 8
9 10 11 12
```

11. Full Program: Print a 2D Array

```
#include <stdio.h>

int main() {
    int A[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            printf("%d ", A[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Expected output:

```
1 2 3 4
5 6 7 8
9 10 11 12
```

12. Printing Only One Row

Suppose we want to print only row 1.

Array:

```
A =  
1  2  3  4  
5  6  7  8  
9 10 11 12
```

Row 1 is:

```
5 6 7 8
```

Code:

```
for (int j = 0; j < 4; j++) {  
    printf("%d ", A[1][j]);  
}
```

Why do we use only one loop?

Because the row is fixed:

```
A[1][j]
```

Only the column changes.

Step-by-step:

```
j = 0 -> A[1][0] = 5  
j = 1 -> A[1][1] = 6  
j = 2 -> A[1][2] = 7  
j = 3 -> A[1][3] = 8
```

Output:

```
5 6 7 8
```

13. Printing Only One Column

Suppose we want to print only column 2.

Array:

```
A =  
1  2  3  4  
5  6  7  8  
9 10 11 12
```

Column 2 is:

```
3  
7  
11
```

Code:

```
for (int i = 0; i < 3; i++) {  
    printf("%d ", A[i][2]);  
}
```

Why do we use only one loop?

Because the column is fixed:

```
A[i][2]
```

Only the row changes.

Step-by-step:

```
i = 0 -> A[0][2] = 3  
i = 1 -> A[1][2] = 7  
i = 2 -> A[2][2] = 11
```

Output:

3 7 11

14. Finding the Sum of All Elements

To find the sum of all elements, we must visit every row and every column.

Code:

```
int sum = 0;

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        sum = sum + A[i][j];
    }
}

printf("Sum = %d", sum);
```

For this array:

1	2	3	4
5	6	7	8
9	10	11	12

Calculation:

$$1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 + 11 + 12 = 78$$

So:

Sum = 78

15. Full Practice Program: Matrix, Row, Column, and Sum

```
#include <stdio.h>

int main() {
    int A[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    int rows = 3;
    int columns = 4;
    int sum = 0;

    printf("Matrix:\n");

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns; j++) {
            printf("%d ", A[i][j]);
        }
        printf("\n");
    }

    printf("\nOnly row 1:\n");
    for (int j = 0; j < columns; j++) {
        printf("%d ", A[1][j]);
    }

    printf("\n\nOnly column 2:\n");
    for (int i = 0; i < rows; i++) {
        printf("%d ", A[i][2]);
    }

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns; j++) {
            sum = sum + A[i][j];
        }
    }

    printf("\n\nSum = %d\n", sum);
}
```

```
    return 0;  
}
```

Expected output:

```
Matrix:  
1 2 3 4  
5 6 7 8  
9 10 11 12  
  
Only row 1:  
5 6 7 8  
  
Only column 2:  
3 7 11  
  
Sum = 78
```

16. Important Memory Truth About 2D Arrays

A 2D array looks like a table to humans.

But memory is still linear.

That means memory stores values in a straight line.

Logical view:

```
1   2   3   4  
5   6   7   8  
9  10  11  12
```

Physical memory view:

```
1 2 3 4 5 6 7 8 9 10 11 12
```

This is one of the most important ideas in Week 4:

A 2D array looks like a table, but memory stores it as a line.

17. Row-Major Mapping

Meaning

Row-major mapping means the array is stored row by row.

Example:

```
A =  
1  2  3  4  
5  6  7  8  
9 10 11 12
```

In row-major order, memory stores the values like this:

```
1 2 3 4 5 6 7 8 9 10 11 12
```

It stores:

```
First row 0  
Then row 1  
Then row 2
```

So:

```
Row 0: 1 2 3 4  
Row 1: 5 6 7 8  
Row 2: 9 10 11 12
```

becomes:

```
1 2 3 4 | 5 6 7 8 | 9 10 11 12
```

C and C++ built-in 2D arrays use row-major order.

18. Row-Major Address Formula

The row-major formula is:

```
Address of A[i][j] = L0 + ((i * N) + j) * w
```

Where:

```
L0 = base address  
i = row index  
j = column index  
N = total number of columns  
w = word size
```

Important:

```
In row-major formula, use total columns.
```

Because row-major skips full rows first.

Each full row contains `N` columns.

19. Understanding the Row-Major Formula

Formula:

```
Address of A[i][j] = L0 + ((i * N) + j) * w
```

Breakdown:

```
i * N = number of elements skipped from previous full rows  
+j     = number of elements moved inside the current row  
*w     = converts element count into byte distance
```

Simple idea:

Skip previous rows.
Then move across the current row.
Then multiply by word size.
Then add the base address.

20. Row-Major Example

Given:

Array: $A[3][4]$
Base address $L0 = 200$
Target element = $A[1][2]$
Total columns $N = 4$
Word size $w = 2$

Find the address of $A[1][2]$ using row-major order.

Formula:

$$\text{Address} = L0 + ((i * N) + j) * w$$

Substitute values:

$$\text{Address} = 200 + ((1 * 4) + 2) * 2$$

Calculate:

$$\begin{aligned}\text{Address} &= 200 + (4 + 2) * 2 \\ \text{Address} &= 200 + 6 * 2 \\ \text{Address} &= 200 + 12 \\ \text{Address} &= 212\end{aligned}$$

Answer:

$$\text{Address of } A[1][2] = 212$$

21. Why We Use `N = 4` in Row-Major

The array is:

```
int A[3][4];
```

So:

```
3 = total rows  
4 = total columns
```

In row-major order, we skip previous rows.

Each full row has 4 elements.

For `A[1][2]`:

```
row index i = 1  
column index j = 2
```

This means:

```
Skip 1 full row.  
Then move 2 columns inside the current row.
```

Row 0 has 4 elements:

```
1 2 3 4
```

Then move inside row 1:

```
A[1][0] = 5  
A[1][1] = 6  
A[1][2] = 7
```

Total elements skipped:

$$4 + 2 = 6$$

Then multiply by word size:

$$6 * 2 = 12 \text{ bytes}$$

Then add base address:

$$200 + 12 = 212$$

22. Column-Major Mapping

Meaning

Column-major mapping means the array is stored column by column.

Example:

```
A =  
1  2  3  4  
5  6  7  8  
9 10 11 12
```

In column-major order, memory stores the values like this:

```
1 5 9 2 6 10 3 7 11 4 8 12
```

It stores:

```
First column 0  
Then column 1  
Then column 2  
Then column 3
```

So:

```
Column 0: 1 5 9
Column 1: 2 6 10
Column 2: 3 7 11
Column 3: 4 8 12
```

becomes:

```
1 5 9 | 2 6 10 | 3 7 11 | 4 8 12
```

Column-major is not the normal built-in 2D array layout in C and C++.

But it is important because it helps us understand general memory mapping.

23. Column-Major Address Formula

The column-major formula is:

```
Address of A[i][j] = L0 + ((j * M) + i) * w
```

Where:

```
L0 = base address
i = row index
j = column index
M = total number of rows
w = word size
```

Important:

```
In column-major formula, use total rows.
```

Because column-major skips full columns first.

Each full column contains **M** rows.

24. Understanding the Column-Major Formula

Formula:

```
Address of A[i][j] = L0 + ((j * M) + i) * w
```

Breakdown:

```
j * M = number of elements skipped from previous full columns
+i     = number of elements moved inside the current column
*w     = converts element count into byte distance
```

Simple idea:

```
Skip previous columns.
Then move down inside the current column.
Then multiply by word size.
Then add the base address.
```

25. Column-Major Example

Given:

```
Array: A[3][4]
Base address L0 = 200
Target element = A[1][2]
Total rows M = 3
Word size w = 2
```

Find the address of `A[1][2]` using column-major order.

Formula:

```
Address = L0 + ((j * M) + i) * w
```

Substitute values:

```
Address = 200 + ((2 * 3) + 1) * 2
```

Calculate:

```
Address = 200 + (6 + 1) * 2  
Address = 200 + 7 * 2  
Address = 200 + 14  
Address = 214
```

Answer:

```
Address of A[1][2] = 214
```

26. Why Row-Major and Column-Major Can Give Different Addresses

For the same logical position:

```
A[1][2]
```

Row-major gave:

```
212
```

Column-major gave:

```
214
```

This happens because the storage order is different.

Row-major stores rows first:

```
1 2 3 4 5 6 7 8 9 10 11 12
```

Column-major stores columns first:

1 5 9 2 6 10 3 7 11 4 8 12

So even though the table position is the same, the memory position can be different.

Important idea:

Same logical element, different memory order, different address.

27. Row-Major vs Column-Major

Feature	Row-Major	Column-Major
Storage direction	Row by row	Column by column
First stored group	First row	First column
Used by C/C++ built-in 2D arrays	Yes	No
Formula	$L0 + ((i * N) + j) * w$	$L0 + ((j * M) + i) * w$
Uses total columns?	Yes	No
Uses total rows?	No	Yes
Skip first	Previous rows	Previous columns

Memory order example:

Array:
1 2 3
4 5 6

Row-major order:

1 2 3 4 5 6

Column-major order:

1 4 2 5 3 6

28. Easy Memory Aid for the Formulas

For row-major:

Rows first.
So row index i is multiplied by total columns.

Formula part:

$i * \text{columns} + j$

Full formula:

$\text{Address} = L0 + ((i * N) + j) * w$

For column-major:

Columns first.
So column index j is multiplied by total rows.

Formula part:

$j * \text{rows} + i$

Full formula:

$\text{Address} = L0 + ((j * M) + i) * w$

Short memory aid:

Row-major: $i * \text{columns} + j$
Column-major: $j * \text{rows} + i$

29. Address Calculation Program in C

This program calculates the row-major and column-major addresses of `A[1][2]`.

```

#include <stdio.h>

int main() {
    int L0 = 200;
    int w = 2;

    int rows = 3;
    int columns = 4;

    int i = 1;
    int j = 2;

    int rowMajorAddress;
    int columnMajorAddress;

    rowMajorAddress = L0 + ((i * columns) + j) * w;
    columnMajorAddress = L0 + ((j * rows) + i) * w;

    printf("Target element: A[%d][%d]\n", i, j);

    printf("\nRow-major calculation:\n");
    printf("Address = L0 + ((i * columns) + j) * w\n");
    printf("Address = %d + ((%d * %d) + %d) * %d\n",
           L0, i, columns, j, w);
    printf("Address = %d\n", rowMajorAddress);

    printf("\nColumn-major calculation:\n");
    printf("Address = L0 + ((j * rows) + i) * w\n");
    printf("Address = %d + ((%d * %d) + %d) * %d\n",
           L0, j, rows, i, w);
    printf("Address = %d\n", columnMajorAddress);

    return 0;
}

```

Expected output:

```

Target element: A[1][2]

Row-major calculation:
Address = L0 + ((i * columns) + j) * w
Address = 200 + ((1 * 4) + 2) * 2
Address = 212

Column-major calculation:
Address = L0 + ((j * rows) + i) * w

```

```
Address = 200 + ((2 * 3) + 1) * 2  
Address = 214
```

30. Three Ways to Create 2D Arrays

2D arrays can be created in different ways.

The syntax may look similar when accessing elements, but the memory structure can be different.

The three important methods are:

1. Normal stack 2D array
2. Array of pointers
3. Double pointer

31. Method 1: Normal Stack 2D Array

Example:

```
int A[3][4];
```

This creates a normal 2D array.

If it is created inside a function, it is usually stored on the stack.

Memory is one continuous block.

Example:

```
#include <stdio.h>  
  
int main() {  
    int A[3][4] = {  
        {1, 2, 3, 4},  
        {5, 6, 7, 8},  
        {9, 10, 11, 12}  
    };  
}
```

```
    printf("%d", A[1][2]);  
  
    return 0;  
}
```

Output:

7

Good for:

Small arrays
Known size
Simple programs

32. Method 2: Array of Pointers

Example:

```
int *B[3];
```

This means:

B is an array of 3 pointers.
Each pointer can point to one row.

Example:

```
#include <stdio.h>  
#include <stdlib.h>  
  
int main() {  
    int *B[3];  
  
    B[0] = (int*)malloc(4 * sizeof(int));  
    B[1] = (int*)malloc(4 * sizeof(int));  
    B[2] = (int*)malloc(4 * sizeof(int));  
  
    B[0][0] = 1;
```

```
B[0][1] = 2;
B[1][0] = 5;
B[1][1] = 6;

printf("%d", B[1][1]);

free(B[0]);
free(B[1]);
free(B[2]);

return 0;
}
```

Output:

6

Important:

Each row may be a separate heap block.

So we should not assume the entire 2D structure is one continuous memory block.

33. Method 3: Double Pointer

Example:

```
int **C;
```

This means:

C points to a pointer array.
Each pointer in that pointer array points to a row.

Example:

```
#include <stdio.h>
#include <stdlib.h>
```

```

int main() {
    int rows = 3;
    int columns = 4;

    int **C = (int**)malloc(rows * sizeof(int*));

    for (int i = 0; i < rows; i++) {
        C[i] = (int*)malloc(columns * sizeof(int));
    }

    C[1][2] = 50;

    printf("%d", C[1][2]);

    for (int i = 0; i < rows; i++) {
        free(C[i]);
    }

    free(C);

    return 0;
}

```

Output:

50

Important:

A double pointer is not automatically the same as a true contiguous 2D array.

The rows may be stored in separate heap blocks.

34. Comparing the Three 2D Array Methods

Method	Example	Main memory idea
Normal 2D array	<code>int A[3][4];</code>	One direct 2D block
Array of pointers	<code>int *B[3];</code>	Stack array of pointers, heap rows
Double pointer	<code>int **C;</code>	Heap pointer array, heap rows

All three may allow syntax like:

```
A[i][j]  
B[i][j]  
C[i][j]
```

But they are not the same in memory.

Important warning:

Same access syntax does not always mean same memory structure.

35. Common Week 4 Mistakes

Mistake 1: Mixing Up Rows and Columns

For:

```
int A[3][4];
```

Correct meaning:

```
3 rows  
4 columns
```

Valid row indexes:

```
0, 1, 2
```

Valid column indexes:

```
0, 1, 2, 3
```

Wrong:

```
A[3][0]
```

Why wrong?

Because row index `3` is outside the array.

The last valid row index is `2`.

Mistake 2: Forgetting the Inner Loop

Wrong:

```
for (int i = 0; i < 3; i++) {  
    printf("%d ", A[i]);  
}
```

This does not correctly print all elements of a 2D array.

Correct:

```
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 4; j++) {  
        printf("%d ", A[i][j]);  
    }  
}
```

A 2D array needs row and column indexes.

Mistake 3: Thinking `A[1][2]` Means First Row, Second Column

Wrong thinking:

```
A[1][2] = first row, second column
```

Correct thinking:

```
A[1][2] = row index 1, column index 2
```

In C/C++:

```
row index 1 = second row  
column index 2 = third column
```

Mistake 4: Using Rows in the Row-Major Formula

Wrong:

```
Address = L0 + ((i * M) + j) * w
```

Correct row-major formula:

```
Address = L0 + ((i * N) + j) * w
```

In row-major mapping:

```
N = total columns
```

Mistake 5: Using Columns in the Column-Major Formula

Wrong:

```
Address = L0 + ((j * N) + i) * w
```

Correct column-major formula:

```
Address = L0 + ((j * M) + i) * w
```

In column-major mapping:

```
M = total rows
```

Mistake 6: Assuming `int **C` Is Always Contiguous

Wrong assumption:

```
int **C is the same as int A[3][4]
```

Correct idea:

```
int A[3][4] is one direct 2D block.  
int **C usually points to separate heap rows.
```

So the memory layout may be different.

36. Key Terms

Term	Simple Meaning
2D array	An array arranged in rows and columns
Row	Horizontal line of values
Column	Vertical line of values
Matrix	Table-like arrangement of values
<code>A[i][j]</code>	Element at row <code>i</code> , column <code>j</code>
Nested loop	A loop inside another loop
Outer loop	Usually controls rows
Inner loop	Usually controls columns
Row-major mapping	Stores elements row by row
Column-major mapping	Stores elements column by column
Base address	Starting address of the array
Word size	Number of bytes used by one element
Total columns <code>N</code>	Number of columns in the array
Total rows <code>M</code>	Number of rows in the array
Stack 2D array	Fixed-size 2D array usually stored on stack
Array of pointers	Array where each element is a pointer to a row

Term	Simple Meaning
Double pointer	Pointer to pointer, often used for dynamic 2D structures

37. Week 4 Formula Summary

Row-Major Formula

Address of $A[i][j] = L0 + ((i * N) + j) * w$

Where:

$L0$ = base address
 i = row index
 j = column index
 N = total columns
 w = word size

Memory aid:

Row-major = row first = $i * \text{columns} + j$

Column-Major Formula

Address of $A[i][j] = L0 + ((j * M) + i) * w$

Where:

$L0$ = base address
 i = row index
 j = column index
 M = total rows
 w = word size

Memory aid:

$$\text{Column-major} = \text{column first} = j * \text{rows} + i$$

38. Week 4 Summary

By the end of Week 4, you should understand that:

- A 2D array is viewed as a table.
 - A 2D array has rows and columns.
 - `A[i][j]` means row `i`, column `j`.
 - C and C++ indexing starts from `0`.
 - A 2D array needs nested loops for full traversal.
 - The outer loop usually controls rows.
 - The inner loop usually controls columns.
 - You can print one row by fixing the row index.
 - You can print one column by fixing the column index.
 - A 2D array looks like a table but is stored linearly in memory.
 - Row-major mapping stores row by row.
 - Column-major mapping stores column by column.
 - C and C++ built-in 2D arrays use row-major order.
 - Row-major formula uses total columns.
 - Column-major formula uses total rows.
 - `int A[3][4]`, `int *B[3]`, and `int **C` may all use `A[i][j]` style access, but their memory structures are different.
-

39. Week 4 Checkpoint Questions

Try answering these without looking at the notes.

1. What is a 2D array?
2. What are rows?
3. What are columns?
4. What does `A[i][j]` mean?
5. In `A[i][j]`, what does `i` usually represent?
6. In `A[i][j]`, what does `j` usually represent?
7. How many rows and columns are in `int A[3][4]`?
8. What are the valid row indexes for `int A[3][4]`?
9. What are the valid column indexes for `int A[3][4]`?
10. Why do we need nested loops for 2D arrays?
11. What does the outer loop usually control?
12. What does the inner loop usually control?
13. How do you print only one row?

14. How do you print only one column?
 15. What does row-major mapping mean?
 16. What does column-major mapping mean?
 17. Which storage order do C and C++ built-in 2D arrays use?
 18. What is the row-major formula?
 19. What does `N` mean in the row-major formula?
 20. What is the column-major formula?
 21. What does `M` mean in the column-major formula?
 22. Why can row-major and column-major give different addresses?
 23. What is the difference between `int A[3][4]` and `int **C`?
 24. Why should we not assume a double pointer is a true contiguous 2D array?
 25. What is the most important idea of Week 4?
-

40. Practice Tasks

Task 1: Print a 3 by 4 Matrix

Write a C program that:

1. Creates a 2D array with 3 rows and 4 columns.
2. Initializes it with values from 1 to 12.
3. Prints it in matrix form.

Expected output:

```
1 2 3 4
5 6 7 8
9 10 11 12
```

Task 2: Print One Row

Using the same array, print only row 1.

Expected output:

```
5 6 7 8
```

Task 3: Print One Column

Using the same array, print only column 2.

Expected output:

```
3 7 11
```

Task 4: Find Sum of All Elements

Using the same array, find the sum of all elements.

Expected answer:

```
78
```

Task 5: Row-Major Address Calculation

Given:

```
L0 = 200  
w = 2  
Array = A[3][4]  
Target = A[1][2]
```

Use row-major formula:

```
Address = L0 + ((i * N) + j) * w
```

Solution:

```
Address = 200 + ((1 * 4) + 2) * 2  
Address = 200 + 6 * 2  
Address = 212
```

Answer:

212

Task 6: Column-Major Address Calculation

Given:

```
L0 = 200  
w = 2  
Array = A[3][4]  
Target = A[1][2]
```

Use column-major formula:

$$\text{Address} = L0 + ((j * M) + i) * w$$

Solution:

```
Address = 200 + ((2 * 3) + 1) * 2  
Address = 200 + 7 * 2  
Address = 214
```

Answer:

214

Task 7: Compare Row-Major and Column-Major

Given:

```
L0 = 500  
w = 4  
Array = A[4][5]  
Target = A[2][3]
```

Row-major:

```
N = 5
Address = 500 + ((2 * 5) + 3) * 4
Address = 500 + 13 * 4
Address = 552
```

Column-major:

```
M = 4
Address = 500 + ((3 * 4) + 2) * 4
Address = 500 + 14 * 4
Address = 556
```

Final answers:

```
Row-major address = 552
Column-major address = 556
```

41. Simple Final Memory Aid

```
1D array = line
2D array = table

A[i] = one index
A[i][j] = two indexes

i = row
j = column

Rows go left to right.
Columns go top to bottom.

Nested loop = loop inside loop.
Outer loop = rows.
Inner loop = columns.

Row-major = row by row.
Column-major = column by column.

Row-major formula:
L0 + ((i * N) + j) * w
```

Column-major formula:
 $L0 + ((j * M) + i) * w$

N = total columns
M = total rows

Most important Week 4 idea:

A 2D array looks like a table, but memory stores it as one straight line.